

Vayita Grow - Development Report & Interview Q&A

The build process, the questions it invites, terms answered, and sample code

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the 58-commit development history

Contents

How to use this report	3
Part 1 - The development story	4
Phase 1 - Foundation	4
Phase 2 - Public site and dashboard shell	4
Phase 3 - The database layer	4
Phase 4 - The data services	4
Phase 5 - Authentication and security	4
Phase 6 - Business features	4
Phase 7 - Hardening	4
Part 2 - Questions and detailed answers	6
“What is Row-Level Security (RLS)?”	6
“What is the dual-source pattern, and why build it?”	6
“Why store the JWT in an httpOnly cookie instead of localStorage?”	6
“How do you revoke a JWT / what is a token blacklist?”	6
“How does TOTP two-factor authentication work?”	6
“What is a connection pool?”	6
“What is a CSP / security headers?”	6
“What are Next.js server actions?”	7
“One codebase for a public site and an internal tool - how did you keep them clean?”	7
Part 3 - The bait-terms dictionary	8
Part 4 - Sample code snippets (pen and paper)	10
The hooks to drop on purpose	12

How to use this report

Vayita Grow is your “digitise how a business runs” project - which is SAP’s exact domain, at small scale. It is a B2B corporate site plus an internal management platform for an agricultural-inputs company, with a genuinely serious backend: real database migrations, JWT sessions, two-factor auth, and role-based access. Structure: **Part 1** build story, **Part 2** Q&A, **Part 3** bait-terms dictionary, **Part 4** sample code.

What it is in one breath: one codebase serving two audiences - a polished public site for dealers and investors, and a secured internal dashboard where staff manage clients, orders, statements, and field reports - built to prove the company runs digitally.

The SAP bridge: “This was literally about digitising how a business runs its operations - the same problem SAP solves, just at small scale.” Say that.

Part 1 - The development story

Phase 1 - Foundation

- Set up the **Next.js App Router** project (TypeScript).
- Built a design foundation - **brand tokens**, typography, and a shared UI component library.
- Laid out the public pages and the dashboard structure.

Phase 2 - Public site and dashboard shell

- Rebuilt the public site with the brand identity and **SEO**.
- Built a **guarded dashboard layout** (only authenticated staff get in) with modules for **catalog**, **sales**, **field ops**, and **system**.

Phase 3 - The database layer

- Added a PostgreSQL layer with the **pg** driver: a **connection pool** and query/list helpers.
- Wrote SQL **schema migrations**: tables, **enums**, **indexes**, a **purge trigger** for old data, and **Row-Level Security (RLS)**.
- A seed script that hashes demo passwords with **Argon2**.
- Introduced a **dual-source** design: a **DATA_SOURCE** flag switches every service between mock data and the real database behind one interface.

Phase 4 - The data services

- Built dual-source services for customers, products and categories, orders and deliveries, inventory, statements and field reports, feedback, activity, settings, and the dashboard - each working the same whether backed by mock data or Postgres.

Phase 5 - Authentication and security

- **jose** JWT sessions stored in **httpOnly cookies**, with roles and a **token blacklist** for revocation.
- Credential verification, **login history**, an **audit trail**, and **rate limiting**.
- **TOTP two-factor auth** via Google Authenticator (**otplib**): enroll, challenge, disable, with a two-step login flow.
- Password **RBAC**: change-own, admin-reset, a strength policy.

Phase 6 - Business features

- **Resend** for contact and public-feedback emails.
- **Cloudinary** for product image uploads.
- A **quotations** module with a branded **PDF export** (jsPDF) and send-to-staff.
- Server-side **Excel exports** (exceljs) for every dashboard module.
- Analytics dashboard on **Recharts** with a chart-type dropdown and PDF export.

Phase 7 - Hardening

- **CSP** and security headers; a Cloudinary image **allowlist**.

- Made the build and sitemap **database-resilient** so a build never needs a live DB.
-

Part 2 - Questions and detailed answers

“What is Row-Level Security (RLS)?”

RLS is a PostgreSQL feature where the database itself enforces which rows a user can see or change, via policies attached to a table - not just the application. So even if application code has a bug, the database will not hand back rows a user is not allowed to. It is defence in depth: the rules live at the data layer, closest to the data.

“What is the dual-source pattern, and why build it?”

Every data service can run against either **mock data** or the **real PostgreSQL database**, chosen by a single `DATA_SOURCE` environment flag, behind one shared interface. That let me build and demo the whole UI with mock data before the database existed, then flip to Postgres without changing any component. It also makes the build independent of a live database, which matters for CI and previews.

“Why store the JWT in an `httpOnly` cookie instead of `localStorage`?”

An `httpOnly` cookie cannot be read by JavaScript, so if the site ever has an XSS bug, the attacker still cannot steal the token - the browser sends it automatically but script can't touch it. `localStorage` is readable by any script on the page, which is a bigger risk. I used jose to sign and verify the JWT, stored it `httpOnly`, and added same-site settings against CSRF.

“How do you revoke a JWT / what is a token blacklist?”

JWTs are stateless, so normally you cannot cancel one before it expires. To support real logout and revocation, each token carries a unique id, and logout adds that id to a **blacklist** that is checked on every request. A blacklisted token is rejected immediately, so a stolen or logged-out token dies at once instead of when the clock says so.

“How does TOTP two-factor authentication work?”

TOTP - Time-based One-Time Password - is what Google Authenticator uses. At enrollment, the server and the app share a secret. The app hashes that secret with the current time (in 30-second windows) to produce a 6-digit code; the server does the same and checks they match. It changes every 30 seconds and needs no network, so even if a password leaks, an attacker cannot log in without the current code.

“What is a connection pool?”

Opening a new database connection per request is slow. A connection pool keeps a set of open connections ready and hands them out, so requests reuse connections instead of paying the setup cost each time. I set a sensible pool size so the app is fast but does not exhaust the database.

“What is a CSP / security headers?”

A Content Security Policy is an HTTP header that tells the browser which sources of scripts, styles, and images are allowed - so an injected script from an unknown origin simply won't run, which blunts

XSS. Alongside it I set headers like HSTS and X-Frame-Options, and an allowlist for the Cloudinary image domain.

“What are Next.js server actions?”

Server actions let a form or component call a server function directly, without me hand-writing a separate API endpoint. The function runs on the server, so I do the auth check and database work there, and validate input before it touches the DB. It keeps sensitive logic off the client.

“One codebase for a public site and an internal tool - how did you keep them clean?”

Route groups separate the public site from the dashboard, and the dashboard sits behind a guarded layout that checks the session before rendering. A shared design system keeps both coherent. So one project, two clearly-separated audiences.

Part 3 - The bait-terms dictionary

Row-Level Security (RLS)

Database-enforced per-row access policies. *Say it:* “RLS enforces access at the data layer, so a code bug can’t leak rows.”

Dual-source (DATA_SOURCE flag)

One interface, switchable between mock and real DB. *Say it:* “A DATA_SOURCE flag flips every service between mock data and Postgres.”

jose / JWT / httpOnly cookie

JWT library; token stored where JS can’t read it. *Say it:* “jose-signed JWT in an httpOnly cookie, so XSS can’t steal it.”

Token blacklist

Revoking JWTs by id on logout. *Say it:* “Each token has an id; logout blacklists it, so it dies immediately.”

TOTP / 2FA / otplib

Time-based one-time codes for a second factor. *Say it:* “TOTP 2FA via Google Authenticator - a 30-second code on top of the password.”

Argon2

Memory-hard, salted password hashing. *Say it:* “Passwords hashed with Argon2, one-way and memory-hard.”

Connection pool

Reused pre-opened DB connections. *Say it:* “A pg connection pool so requests reuse connections instead of reconnecting.”

CSP / security headers

Browser policy limiting allowed sources. *Say it:* “A Content Security Policy plus HSTS and frame headers blunt XSS and clickjacking.”

RBAC

Role-based access control. *Say it:* “Password and dashboard access are role-based.”

Server actions

Server functions callable from the client directly. *Say it:* “Server actions run auth and DB work on the server, off the client.”

Migrations / enums / indexes / purge trigger

Versioned schema changes; typed value sets; lookup speedups; auto-cleanup. *Say it:* “SQL migrations with enums, indexes, and a purge trigger for old rows.”

Recharts / jsPDF / exceljs

Charting, PDF export, Excel export libraries. *Say it:* “Recharts dashboards, with jsPDF and exceljs exports.”

Audit trail / login history

Recording who did what and when. *Say it:* “An audit trail and login history for accountability.”

Part 4 - Sample code snippets (pen and paper)

jose JWT session in an httpOnly cookie

```
import { SignJWT } from "jose";
import { cookies } from "next/headers";

async function login(user, secret: Uint8Array) {
  const token = await new SignJWT({ sub: user.id, role: user.role })
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime("2h")
    .sign(secret);
  cookies().set("session", token, { httpOnly: true, sameSite: "lax", secure: true });
}
```

Say aloud: “Signed with jose, stored httpOnly so JavaScript - and any XSS - can’t read it.”

A Row-Level Security policy (SQL)

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY staff_sees_own ON orders
  FOR SELECT
  USING (created_by = current_setting('app.user_id')::int);
```

Say aloud: “The database itself only returns rows this user created - access enforced at the data layer.”

The dual-source service pattern

```
const useDb = process.env.DATA_SOURCE === "db";

export async function getCustomers() {
  return useDb ? db.list("SELECT * FROM customers") : mock.customers;
}
```

Say aloud: “One interface; a flag switches the whole app between mock data and Postgres.”

Verify a TOTP 2FA code (otplib)

```
import { authenticator } from "otplib";

function verify2FA(userSecret: string, code: string) {
  return authenticator.verify({ token: code, secret: userSecret });
}
```

Say aloud: “The 6-digit code is checked against the shared secret and current time window.”

A guarded server action

```
"use server";
export async function deleteOrder(id: number) {
  const session = await getSession();           // read httpOnly cookie
  if (!session || session.role !== "admin") throw new Error("forbidden");
  await db.query("DELETE FROM orders WHERE id = $1", [id]); // parametrised
}
```

Say aloud: “Auth is checked on the server, and the query is parametrised - no SQL injection.”

The hooks to drop on purpose

1. Row-Level Security (database-enforced access)
2. Dual-source DATA_SOURCE architecture
3. JWT in httpOnly cookies + token blacklist for revocation
4. TOTP two-factor auth
5. Parametrised queries / server-side auth in server actions
6. CSP and security headers

And always land the SAP bridge: “digitising how a business runs is exactly what SAP does.”